Problem A

Your colleague Nathália has a training scheduled to take place in person at SAP Labs and needs your help finding the room for her appointment. She received a code with 4 characters (GE02) that represents the location of the room.

You know that there are 4 possible floors, which are: (B=basement, G=ground, 1=first, 2=second). In addition, there are 10 sectors represented by letters of the alphabet from A to J. Finally, the rooms numbers are integers ranging from 01 to 20.

Therefore, help your colleague by creating a program that receives the room code and prints its location in the pattern “Room < room_number > on the < floor > floor of sector < sector >”.

Inputs

The input contains 4 characters, where the first character represents the floor, the second represents the sector of the building and the last two represent the room number.

Outputs

For each input test case, your program must print a single line containing the location of the room in text format.

Examples

Sample Input 1	Sample Output 1
GE02	Room 02 on the ground floor of sector E

Sample Input 2	Sample Output 2
2F06	Room 06 on the second floor of sector F

Code

```python
# Estrutura de dados utilizada: dicionário para mapear o código do andar para o nome do andar
# Tipo de algoritmo: extração e formatação de string

floor_map = {
    'B': 'basement',
    'G': 'ground',
    '1': 'first',
    '2': 'second'
}

code = input().strip()
floor_code = code[0]
sector = code[1]
room_number = code[2:]

floor = floor_map.get(floor_code, floor_code)
print(f"Room {room_number} on the {floor} floor of sector {sector}")
```

Problem B

Com a data para o novo formato híbrido aproximando-se, a SAP decidiu construir um novo edifício para comportar os funcionários que passarão a vir mais vezes ao escritório. No projeto de construção deste edifício, existem ao todo N salas de trabalho, conectadas por corredores, de tal forma que é possível trafegar de uma sala para qualquer outra seguindo um caminho entre salas e corredores. Para cada corredor que conecta duas salas, existe um custo Ci para a construção do corredor.

Sua tarefa é determinar qual o menor custo necessário para construir o edifício, dada a configuração do projeto inicial, e a restrição de que na construção final a regra de poder ir de uma sala para outra qualquer passando por uma sequência de salas e corredores deve ser mantida. O custo da construção de uma sala é negligível, apenas o custo da construção dos corredores importa.

Inputs

A primeira linha da entrada contém dois inteiros, N e M ($2 \leq N \leq 103$ e $1 \leq M \leq 105$), sendo o número de salas existentes no projeto do edifício, e o número de corredores, respectivamente. As salas são enumeradas de 1 a N.

Cada uma das M linhas a seguir contém três inteiros Ai, Bi e Ci ($1 \leq Ci \leq 103$), indicando que no projeto inicial do edifício existe um corredor que conecta as salas Ai e Bi e que seu custo de construção é Ci.

Outputs

Seu algoritmo deve imprimir uma única linha com um número inteiro, indicando o menor custo possível para a construção do edifício, dadas as restrições impostas.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4 4	60
1 2 10	
1 3 40	
2 4 30	
3 4 20	

Exemplo de entrada 2	Exemplo de saída 2
5 6	
1 2 8	
2 3 12	
2 4 9	
2 5 3	

Exemplo de entrada 2	Exemplo de saída 2
3 4 5	
4 5 1	

Code

```python
# Estrutura de dados utilizada: lista de arestas e estrutura de união-busca (Union-Find)
# Tipo de algoritmo: Algoritmo de Kruskal para Árvore Geradora Mínima (MST)

def find(parent, x):
    if parent[x] != x:
        parent[x] = find(parent, parent[x])
    return parent[x]

def union(parent, rank, x, y):
    xroot = find(parent, x)
    yroot = find(parent, y)
    if xroot == yroot:
        return False
    if rank[xroot] < rank[yroot]:
        parent[xroot] = yroot
    else:
        parent[yroot] = xroot
        if rank[xroot] == rank[yroot]:
            rank[xroot] += 1
    return True

N, M = map(int, input().split())
edges = []
for _ in range(M):
    a, b, c = map(int, input().split())
    edges.append((c, a-1, b-1))

edges.sort()
parent = [i for i in range(N)]
rank = [0] * N
cost = 0
for c, a, b in edges:
    if union(parent, rank, a, b):
        cost += c

print(cost)
```

Problem C

João está prestes a completar dois anos de contrato na SAP e está bem feliz com seu trabalho. Ele gosta tanto de trabalhar que acabou esquecendo de marcar suas férias e, por isso, recebeu uma notificação da sua gestora que ele precisa fazer isso o quanto antes! Contudo, João só pode tirar férias se a data de início e fim não sobrepor as datas de nenhum outro colega de trabalho.

Será que João vai conseguir tirar suas férias?

Inputs

A primeira linha de entrada consiste em um inteiro N (0 <= N <= 104) representando a quantidade de colegas que já marcaram suas férias. Em seguida, seguem N linhas, cada uma contendo a data de início e de fim das férias do i-ésimo colega. A data é representada por uma string no formato dd-mm, onde dd é o dia e mm o mês. Considere que todas as datas fornecidas são válidas. A última linha consiste na data de início e de fim que João gostaria de tirar suas férias.

Outputs

Seu programa deve imprimir uma única linha com a palavra “Sim”, caso João possa tirar férias ou “Nao”, caso contrário.

Examples

Exemplo de entrada 1	**Exemplo de saída 1**
0	Sim
10-10 17-10	

Exemplo de entrada 2	**Exemplo de saída 2**
1	Nao
10-10 17-10	
17-10 28-10	

Code

```python
# Estrutura de dados utilizada: lista de tuplas para armazenar intervalos de datas (interval scheduling)
# Algoritmo: verificação de sobreposição de intervalos

def to_day_of_year(date):
    # Dias acumulados até o início de cada mês (não bissexto)
    days_in_month = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334]
    day, month = map(int, date.split('-'))
    return days_in_month[month - 1] + day

N = int(input())
intervals = []
for _ in range(N):
    start, end = input().split()
    intervals.append((to_day_of_year(start), to_day_of_year(end)))

joao_start, joao_end = input().split()
joao_start = to_day_of_year(joao_start)
joao_end = to_day_of_year(joao_end)

possible = True
for start, end in intervals:
    # Verifica se há sobreposição de intervalos
    if not (joao_end < start or joao_start > end):
        possible = False
        break

print("Sim" if possible else "Nao")
```

Problem D

A equipe de RH da SAP teve uma ideia de atrair novos talentos e promoveu um evento para os participantes da Maratona de Programação. Com isso, foram organizadas palestras, tours pelo Labs, entre outras atividades. E no meio de tudo isso, será feita uma pausa para o lanche, ou coffee break. Foram disponibilizadas várias opções, incluindo sanduíches em formatos circulares e triangulares, representados por 0 e 1, respectivamente. As pessoas que optarem pelo sanduíche, deverão formar uma fila e esperar a sua vez. Os sanduíches estão organizados como se fossem uma pilha. A ordem para pegar um sanduíche é a seguinte:

- Se a primeira pessoa da fila optou pelo tipo de sanduíche que está no topo da pilha, ele pega o sanduíche e sai da fila.
- Se não é o sanduíche que ele optou, ele não pega o sanduíche e volta para o final da fila.

O processo continua até que nenhum dos participantes possa pegar o sanduíche que está no topo da pilha e, com isso, não comer um sanduíche. Sua tarefa é dizer quantos participantes não conseguiram pegar um sanduíche.

Inputs

A primeira linha de entrada consiste em um inteiro N (1 <= N <= 100), representando a quantidade de participantes e a quantidade de sanduíches. Na segunda linha, seguem N inteiros, 0 e 1, separados por espaço, descrevendo a opção do i-ésimo participante. Na última linha, seguem N inteiros, também 0 e 1, separados por espaço, descrevendo o j-ésimo tipo de sanduíche na pilha.

Considere que o início da fila é o participante na posição i = 0 e no topo da pilha está o sanduíche na posição j = 0.

Outputs

A saída consiste em uma única linha contendo um único inteiro, com a quantidade de participantes que não conseguiram pegar um sanduíche.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4	0
1 1 0 0	
0 1 0 1	

Exemplo de entrada 2	Exemplo de saída 2
6	3
1 1 1 0 0 1	
1 0 0 0 1 1	

Code

```python
# Estruturas de dados utilizadas: fila (queue) para os participantes e lista (stack) para os sanduíches
# Tipo de algoritmo: simulação do processo com fila e pilha

from collections import deque

N = int(input())
fila = deque(map(int, input().split()))
pilha = list(map(int, input().split()))

tentativas = 0
while fila and pilha:
    if fila[0] == pilha[0]:
        fila.popleft()
        pilha.pop(0)
        tentativas = 0
    else:
        fila.append(fila.popleft())
        tentativas += 1
        if tentativas == len(fila):
            break  # Ninguém quer o sanduíche do topo

print(len(fila))
```

Problem E

João mora em Viamão e todo dia útil precisa acordar bem cedo para pegar o SAP Bus. Geralmente, ele chega no horário previsto, bem a tempo de participar da primeira reunião do dia. Como alguns trechos da BR estão em obra, em alguns dias ele se atrasa. Para chegar até o seu local de trabalho, João precisa fazer uma boa caminhada pelo Labs, além de pegar um elevador, visto que seu time trabalha no último andar do prédio.

Não bastasse estar atrasado, em um dia desses, o elevador estava com defeito e João precisou subir as escadas. Do térreo até o último andar, é necessário subir N degraus. Além disso, cada degrau infringe um cansaço Ci, sendo i o i-ésimo degrau da escadaria. João pode subir 1 ou 2 degraus de uma só vez, podendo começar tanto no degrau 0 quanto no degrau 1. Sua tarefa é minimizar o cansaço de João para que ele não chegue suado na reunião.

Inputs

A primeira linha de entrada consiste em um inteiro N (2 <= N <= 1000), representando o número de degraus. Na linha seguinte, seguem N inteiros Ci (0 <= Ci <= 999 e 1 <= i <= N), separados por espaço, representando o cansaço atribuído ao degrau Ci.

Outputs

A saída consiste em uma única linha contendo um único inteiro, representando o mínimo de cansaço necessário para subir todos os degraus.

Examples

Exemplo de entrada 1	Exemplo de saída 1
3 10 15 20	15

Exemplo de entrada 2	Exemplo de saída 2
10 1 100 1 1 1 100 1 1 100 1	6

Code

```python
# Estrutura de dados utilizada: lista para programação dinâmica (DP)
# Tipo de algoritmo: Programação Dinâmica (Dynamic Programming)

N = int(input())
C = list(map(int, input().split()))

dp = [0] * N
dp[0] = C[0]
dp[1] = C[1]

for i in range(2, N):
    dp[i] = C[i] + min(dp[i-1], dp[i-2])

print(min(dp[N-1], dp[N-2]))
```

Problem F

Hoje o seu time de trabalho na SAP decidiu fazer a reunião da daily de uma forma diferente! Os membros do time estão espalhados pelo labs, cada um em um local diferente, e vocês decidiram que a daily vai ser feita a partir dos coffee-corners!

Para isso, cada membro do time irá se deslocar até o seu coffee-corner mais próximo e participar da reunião remotamente. Além disso, foi combinado que o primeiro membro a chegar em um coffee-corner será o que vai compartilhar a tela do seu notebook e conduzir a daily.

Dadas as coordenadas (x, y) dos membros do time e dos coffee-corners, a sua tarefa é descobrir qual é este membro que chegará primeiro em um coffee-corner. Cada pessoa consegue mover-se somente nas quatro direções (para cima, para baixo, para a esquerda, para a direita), uma unidade de cada vez. Cada membro do time irá mover-se no menor caminho possível até o coffee-corner mais próximo de si, sendo que todos iniciam a caminhada ao mesmo tempo, e caminham na mesma velocidade. É garantido que sempre há um único membro do time que chegará primeiro a um coffee-corner.

Inputs

A primeira linha da entrada contém dois inteiros, N ($1 \leq N \leq 103$) e M ($1 \leq M \leq 103$), sendo a quantidade de membros do time e de coffee-corners, respectivamente. As N linhas seguintes contêm dois inteiros cada, Ax e Ay ($0 \leq Ax$, $Ay \leq 105$), as coordenadas do i-ésimo membro. As M linhas seguintes também contêm dois inteiros cada, $Bx \geq 0$ e $By \geq 0$ ($0 \leq Bx$, $By \leq 105$), as coordenadas do i-ésimo coffee-corner.

Outputs

Seu programa deve imprimir uma única linha contendo um número inteiro, indicando o membro do time que chegará primeiro a um coffe-corner.

Examples

Exemplo de entrada 1	Exemplo de saída 1
3 2 1 1 5 1 10 3 2 2 5 8	1

Exemplo de entrada 2	Exemplo de saída 2
1 1 0 3	1

Exemplo de entrada 2	Exemplo de saída 2
0 0	

Code

```python
# Estrutura de dados utilizada: listas para armazenar as coordenadas dos membros e dos coffee-corners
# Tipo de algoritmo: busca exaustiva (brute-force) com cálculo de distância de Manhattan

N, M = map(int, input().split())
membros = [tuple(map(int, input().split())) for _ in range(N)]
cafes = [tuple(map(int, input().split())) for _ in range(M)]

menor_tempo = float('inf')
vencedor = -1

for i, (mx, my) in enumerate(membros):
    tempo = min(abs(mx - cx) + abs(my - cy) for (cx, cy) in cafes)
    if tempo < menor_tempo:
        menor_tempo = tempo
        vencedor = i + 1  # índice começa em 1 conforme o problema

print(vencedor)
```

Problem G

Na SAP você pode agendar uma massagem de 10 minutos toda semana. As vagas são limitadas, sendo que o primeiro horário inicia às 14:00 e o último termina às 18:00, de segunda a sexta. A inscrição é disponibilizada semanalmente às 13h da segunda-feira.

Nas últimas semanas têm sido difíceis conseguir um horário, pois as vagas são preenchidas muito rapidamente. Devido a isto, você precisa criar um programa para encontrar o primeiro horário disponível, desde que você não tenha outro compromisso na agenda.

Inputs

A primeira linha contém dois números inteiros e1 e e2 ($0 \leq e1, e2 \leq 100$) representando, respectivamente, o número de eventos da sua agenda pessoal e o número de horários de massagens disponíveis. As próximas e1 linhas são compostas por três informações: dia da semana d (2=segunda, 3=terça, 4=quarta, 5=quinta e 6=sexta), horário de início h (HH:MM, onde HH=horas e MM=minutos) e a duração em minutos m (15, 30, 60, 90, 120, 180). As próximas e2 linhas são compostas por duas informações: d (dia da semana) e h (horário de início).

Outputs

Para cada caso de teste, seu programa deve imprimir uma linha única contendo o primeiro horário da massagem e o dia da semana disponível no padrão "HH:MM na dia_da_semana". Porém, caso não tenha disponibilidade, imprima a mensagem "Sem massagem essa semana 😦".

Examples

Exemplo de entrada 1	Exemplo de saída 1
2 4	17:50 na quinta
2 15:00 30	
4 14:00 60	
2 15:20	
4 14:40	
5 17:50	
6 16:10	

Exemplo de entrada 2	Exemplo de saída 2
1 4	Sem massagem essa semana 😦
6 15:00 180	
6 16:20	

Exemplo de entrada 2	Exemplo de saída 2
6 16:30	
6 17:00	
6 17:10	

Code

```python
# Estrutura de dados utilizada: listas para armazenar eventos e horários de massagem
# Tipo de algoritmo: verificação de sobreposição de intervalos (interval overlap checking)

def time_to_minutes(t):
    h, m = map(int, t.split(':'))
    return h * 60 + m

dias_semana = {2: 'segunda', 3: 'terça', 4: 'quarta', 5: 'quinta', 6: 'sexta'}

e1, e2 = map(int, input().split())
eventos = []
for _ in range(e1):
    d, h, m = input().split()
    d = int(d)
    ini = time_to_minutes(h)
    fim = ini + int(m)
    eventos.append((d, ini, fim))

massagens = []
for _ in range(e2):
    d, h = input().split()
    d = int(d)
    ini = time_to_minutes(h)
    fim = ini + 10
    massagens.append((d, ini, fim, h))

# Ordena por dia e horário
massagens.sort()

def livre(massagem):
    d, ini, fim, _ = massagem
    for ed, eini, efim in eventos:
        if d == ed and not (fim <= eini or ini >= efim):
            return False
    return True

for massagem in massagens:
    if livre(massagem):
        print(f"{massagem[3]} na {dias_semana[massagem[0]]}")
        break
else:
    print("Sem massagem essa semana :(")
```

Problem H

Juliano é fã do programa de auditório Apagando e Ganhando, um programa no qual os participantes são selecionados através de um sorteio e recebem prêmios em dinheiro por participarem. No programa, o apresentador escreve um número de N dígitos em uma lousa. O participante então deve apagar exatamente D dígitos do número que está na lousa; o número formado pelos dígitos que restaram é então o prêmio do participante. Juliano finalmente foi selecionado para participar do programa, e pediu que você escrevesse um programa que, dados o número que o apresentador escreveu na lousa, e quantos dígitos Juliano tem que apagar, determina o valor do maior prêmio que Juliano pode ganhar.

Inputs

A entrada contém vários casos de teste. A primeira linha de cada caso de teste contém dois inteiros N e D ($1 \leq D < N \leq 105$), indicando a quantidade de dígitos do número que o apresentador escreveu na lousa e quantos dígitos devem ser apagados. A linha seguinte contém o número escrito pelo apresentador, que não contém zeros à esquerda. O final da entrada é indicado por uma linha que contém apenas dois zeros, separados por um espaço em branco.

Outputs

Para cada caso de teste da entrada seu programa deve imprimir uma única linha na saída, contendo o maior prêmio que Juliano pode ganhar.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4 2	79
759	323
6 3	100
123123	
7 4	
1000000	
0 0	
Exemplo de entrada 2	**Exemplo de saída 2**

Code

```python
# Estrutura de dados utilizada: pilha (stack)
# Tipo de algoritmo: algoritmo guloso (greedy) com pilha para remoção de dígitos

while True:
    line = input().strip()
    if line == "0 0":
        break
    N, D = map(int, line.split())
    num = input().strip()
    stack = []
    to_remove = D
    for digit in num:
        while stack and to_remove > 0 and stack[-1] < digit:
            stack.pop()
            to_remove -= 1
        stack.append(digit)
    # Se ainda restam dígitos para remover, remova do final
    result = ''.join(stack[:N-D])
    print(result)
```

Problem I

Felipinho está empolgado com seu novo jogo de RPG sobre guerras entre clãs de vampiros. Nesse jogo ele representa um personagem de um vampiro e constantemente entra em conflito contra vampiros de outros clãs. Tais batalhas são realizadas com base nas características de cada personagem envolvido e com a ajuda de um dado comum de seis faces. Por simplicidade, vamos considerar apenas as lutas entre dois vampiros, vampiro 1 e vampiro 2. Cada um possui uma energia vital (chamaremos de EV1 e EV2 respectivamente) e, além disso, são determinadas uma força de ataque AT e uma capacidade de dano D. O combate é realizado em turnos da maneira descrita a seguir. A cada turno um dado é rolado, se o valor obtido for menor do que ou igual a AT, o vampiro 1 venceu o turno, caso contrário o vampiro 2 é quem venceu. O vencedor suga energia vital do adversário igual ao valor D, ou seja, D pontos de EV são diminuídos do perdedor e acrescentados ao vencedor. O combate segue até que um dos vampiros fique com EV igual a ou menor do que zero. Por exemplo, suponhamos que EV1 = 7, EV2 = 5, AT = 2, D = 4. Rola-se o dado e o valor obtido foi 3. Nesse caso, o vampiro 2 venceu o turno e, portanto, 4 pontos de EV são diminuídos do vampiro 1 e acrescentados ao vampiro 2. Sendo assim, os novos valores seriam EV1 = 3 e EV2 = 9. Observe que se no próximo turno o vampiro 2 ganhar novamente, o combate irá terminar. Os valores de AT e D são constantes durante todo o combate, apenas EV1 e EV2 variam. Apesar de gostar muito do jogo, Felipinho acha que os combates estão muito demorados e gostaria de conhecer de antemão a probabilidade de vencer, para saber se vale a pena lutar. Assim, ele pediu que você escrevesse um programa que, dados os valores iniciais de EV1, EV2, além de AT e D, calculasse a probabilidade de o vampiro 1 vencer o combate.

Inputs

A entrada contém vários casos de teste. Cada caso de teste ocupa uma única linha, com os quatro inteiros EV1, EV2, AT e D separados por um espaço em branco ($1 \leq EV1, EV2 \leq 10$, $1 \leq AT \leq 5$ e $1 \leq D \leq 10$). O final da entrada é indicado por uma linha contendo quatro zeros.

Outputs

Para cada caso de teste da entrada seu programa deve imprimir uma única linha. A linha deve conter apenas um número real com precisão de uma casa decimal, representando, em termos de percentagem, a probabilidade de o vampiro 1 vencer o combate.

Examples

Exemplo de entrada 1	Exemplo de saída 1
1 1 3 1	50.0
1 2 1 1	3.2
8 5 3 1	61.5
7 5 2 4	20.0
0 0 0 0	

Code

```python
# Estrutura de dados utilizada: dicionário para memoização (programação dinâmica)
# Tipo de algoritmo: Programação dinâmica recursiva com memoização

def prob_vitoria(ev1, ev2, at, d, memo):
    if ev1 <= 0:
        return 0.0
    if ev2 <= 0:
        return 1.0
    key = (ev1, ev2)
    if key in memo:
        return memo[key]
    p1 = at / 6.0
    p2 = 1 - p1
    # Vampiro 1 vence o turno
    win1 = prob_vitoria(ev1 + d, ev2 - d, at, d, memo)
    # Vampiro 2 vence o turno
    win2 = prob_vitoria(ev1 - d, ev2 + d, at, d, memo)
    memo[key] = p1 * win1 + p2 * win2
    return memo[key]

while True:
    line = input().strip()
    if line == "0 0 0 0":
        break
    EV1, EV2, AT, D = map(int, line.split())
    memo = {}
    prob = prob_vitoria(EV1, EV2, AT, D, memo)
    print(f"{prob * 100:.1f}")
```

Problem J

Um loop musical é um trecho de música que foi composto para repetir continuamente (ou seja, o trecho inicia novamente toda vez que chega ao final), sem que se note descontinuidade. Loops são muito usados na sonorização de jogos, especialmente jogos casuais pela internet. Loops podem ser digitalizados por exemplo utilizando PCM. PCM, do inglês Pulse Code Modulation, é uma técnica para representação de sinais analógicos, muito utilizada em áudio digital. Nessa técnica, a magnitude do sinal é amostrada a intervalos regulares de tempo, e os valores amostrados são armazenados em sequência. Para reproduzir a forma de onda amostrada, o processo é invertido (demodulação). Fernandinha trabalha para uma empresa que desenvolve jogos e compôs um bonito loop musical, codificando-o em PCM. Analisando a forma de onda do seu loop em um software de edição de áudio, Fernandinha ficou curiosa ao notar a quantidade de "picos" existentes. Um pico em uma forma de onda é um valor de uma amostra que representa um máximo ou mínimo local, ou seja, um ponto de inflexão da forma de onda. A figura abaixo ilustra (a) um exemplo de forma de onda e (b) o loop formado com essa forma de onda, contendo 48 picos. Fernandinha é uma amiga muito querida e pediu sua ajuda para determinar quantos picos existem no seu loop musical.

Inputs

A entrada contém vários casos de teste. A primeira linha de um caso de teste contém um inteiro N, representando o número de amostras no loop musical de Fernandinha ($2 \le N \le 104$). A segunda linha contém N inteiros Hi, separados por espaços, representando a sequência de magnitudes das amostras ($-104 \le Hi \le 104$ para $1 \le i \le N$, H1 □ HN e Hi □ Hi+1 para $1 \le i < N$). Note que H1 segue HN quando o loop é reproduzido. O final da entrada é indicado por uma linha que contém apenas o número zero.

Outputs

Para cada caso de teste da entrada seu programa deve imprimir uma única linha, contendo apenas um inteiro, o número de picos existentes no loop musical de Fernandinha.

Examples

Exemplo de entrada 1	Exemplo de saída 1
2	2
1 -3	2
6	4
40 0 -41 0 41 42	
4	
300 450 449 450	
0	

Code

```python
# Estrutura de dados utilizada: lista para armazenar as amostras
# Tipo de algoritmo: varredura sequencial (linear scan) com comparação de vizinhos

while True:
    N = int(input())
    if N == 0:
        break
    H = list(map(int, input().split()))
    picos = 0
    for i in range(N):
        prev = H[i-1]
        curr = H[i]
        next = H[(i+1)%N]
        if (curr > prev and curr > next) or (curr < prev and curr < next):
            picos += 1
    print(picos)
```

Problem K

A Sociedade de Balões Coloridos (SBC) é a principal fornecedora de balões para competições de programação; ela dispõe de grandes fábricas e depósitos, além de uma extensa frota de caminhões para garantir a alegria dos competidores. Há várias sedes regionais na Nlogônia, todas as quais contrataram a SBC para o fornecimento de balões para a prova. A Nlogônia é um arquipélago ligado por várias pontes. Cada ilha do arquipélago pode conter várias sedes regionais e vários depósitos da SBC. Ao planejar as rotas, a SBC se deparou com um problema: por razões de segurança, cada ponte da Nlogônia tem um limite máximo de peso permitido para os veículos que trafegam sobre ela. Devido ao grande número de pontes na Nlogônia, e ao elevado peso da mercadoria transportada, o diretor de operações da SBC pediu que você escrevesse um programa que determina o maior peso bruto que pode ser transportado entre os depósitos e os locais de prova.

Inputs

A primeira linha contém três inteiros N, M e S, indicando, respectivamente, o número de ilhas da Nlogônia, o número de pontes que ligam as ilhas e o número de sedes. As ilhas nlogonianas são numeradas de 1 a N. Cada uma das M linhas seguintes descreve uma ponte. A descrição de cada ponte consiste de uma linha contendo três inteiros A, B e P, indicando as duas ilhas ligadas por aquela ponte e o peso máximo permitido naquela ponte, em toneladas. Todas as pontes são de mão dupla; cada par de ilhas é ligado por no máximo uma ponte; é possível ir de qualquer ilha para qualquer outra ilha utilizando apenas as pontes do arquipélago (mas pode ser preciso passar por outras ilhas primeiro). Cada uma das S linhas seguintes descreve uma sede. A descrição de cada sede consiste de uma linha contendo dois inteiros A e B, indicando, respectivamente, a ilha onde está a sede e a ilha onde está o depósito que irá fornecer os balões àquela sede.

RESTRIÇÕES

- $2 \leq N \leq 2 \times 104$
- $1 \leq M \leq 105$
- $1 \leq S \leq 5 \times 104$
- $1 \leq A, B \leq N, A \mathrel{!=} B$
- $0 \leq P \leq 105$

Outputs

Para cada sede, na ordem em que elas foram descritas na entrada, seu programa deve imprimir uma linha contendo um único inteiro, indicando o maior peso bruto, em toneladas, que pode ser transportado por caminhão do depósito que irá fornecer os balões até ela.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4 5 4	7
1 2 9	9

Exemplo de entrada 1	Exemplo de saída 1
1 3 0	8
2 3 8	7
2 4 7	
3 4 4	
1 4	
2 1	
3 1	
4 3	

Exemplo de entrada 2	Exemplo de saída 2
4 5 2	20
1 2 30	40
2 3 20	
3 4 10	
4 1 40	
2 4 50	
1 3	
1 2	

Code

```python
# Estrutura de dados utilizada: lista de adjacência para o grafo, fila de prioridade (heap) para busca
# Tipo de algoritmo: Algoritmo de caminho máximo mínimo (Widest Path) usando Dijkstra modificado

import sys
import heapq

input = sys.stdin.readline

N, M, S = map(int, input().split())
graph = [[] for _ in range(N+1)]
for _ in range(M):
    A, B, P = map(int, input().split())
    graph[A].append((B, P))
    graph[B].append((A, P))

queries = [tuple(map(int, input().split())) for _ in range(S)]

def widest_path(start, end):
    visited = [False] * (N+1)
    max_weight = [0] * (N+1)
    max_weight[start] = float('inf')
    heap = [(-max_weight[start], start)]
    while heap:
        w, u = heapq.heappop(heap)
        w = -w
        if visited[u]:
            continue
        visited[u] = True
        if u == end:
            return w
        for v, p in graph[u]:
            if not visited[v]:
                new_w = min(w, p)
                if new_w > max_weight[v]:
                    max_weight[v] = new_w
                    heapq.heappush(heap, (-new_w, v))
    return 0

for sede, deposito in queries:
    print(widest_path(deposito, sede))
```

Problem L

Uma das principais dificuldades de organizar uma Maratona de Programação é recolher os balões que escapam e ficam presos no teto do salão: muitas vezes o contrato com o dono do salão exige que este seja entregue limpo logo após o evento, sob pena de multa. Este ano a organização da Maratona está mais previdente: ela tem o desenho do teto do salão, e quer sua ajuda para determinar o que pode acontecer com um balão, dependendo da posição no solo onde ele é solto (isto é, se é bloqueado pelo teto ou se escapa para o exterior do salão). O teto do salão é formado por vários planos que, vistos de lado, podem ser descritos por segmentos de reta.

O balão pode ser considerado pontual. Quando um balão toca um segmento do teto que é horizontal, ele fica preso. Quando um balão toca um segmento que é inclinado, o balão desliza até o ponto mais alto do segmento e escapa, podendo escapar completamente do salão ou podendo tocar em mais segmentos. Não há pontos em comum entre os segmentos que formam o teto. Por exemplo, se o balão for solto nas posições marcadas como a ou b, será bloqueado na posição de coordenadas (2, 5); se o balão for solto na posição marcada como c, será bloqueado na posição de coordenadas (6, 5); e se o balão for solto na posição marcada como d, não será bloqueado e escapará para fora do salão na posição de coordenada x = 7. Escreva um programa que, dada a descrição do teto do salão como segmentos de reta, responde a uma série de consultas sobre a posição final de balões soltos do piso do salão.

Inputs

A primeira linha da entrada contém dois inteiros N e C indicando, respectivamente, o número de segmentos de reta do teto e o número de consultas. Cada uma das N linhas seguintes contém quatro inteiros X1, Y1, X2, Y2, descrevendo um segmento de reta do perfil do teto, com extremos de coordenadas (X1, Y1) e (X2, Y2). Cada uma das C linhas seguintes descreve uma consulta e contém um inteiro X, indicando que a consulta quer determinar o que acontece com um balão solto no ponto de coordenada (X, 0).

RESTRIÇÕES

- $1 \leq N \leq 105$
- $1 \leq C \leq 105$
- $0 \leq X1, X2 \leq 106$, $0 < Y1, Y2 \leq 106$, $X1 \mathrel{!=} X2$
- não há dois valores de coordenadas x iguais, considerando todos os segmentos.
- $0 \leq X \leq 106$

Outputs

Para cada consulta da entrada, seu programa deve imprimir uma única linha. Se o balão escapar do salão, a linha deve conter um único inteiro X, indicando a coordenada x pela qual o balão escapa do salão. Caso contrário, a linha deve conter dois inteiros X e Y indicando a posição (x, y) em que o balão fica retido no teto.

Examples

Exemplo de entrada 1	Exemplo de saída 1

Exemplo de entrada 1	Exemplo de saída 1
4 4	2 5
0 1 3 3	2 5
1 5 6 5	7
5 3 2 4	6 5
7 4 10 2	
2	
5	
8	
6	

Exemplo de entrada 2	Exemplo de saída 2
4 3	1
1 3 4 2	7
10 3 7 4	8 3
2 3 8 3	
3 5 5 4	
4	
9	
8	

Code

```python
# Estrutura de dados utilizada: lista de segmentos ordenada por intervalo x, busca binária para consultas
# Tipo de algoritmo: busca binária e simulação

import sys
import bisect

input = sys.stdin.readline

N, C = map(int, input().split())
segments = []
for _ in range(N):
    x1, y1, x2, y2 = map(int, input().split())
    if x1 > x2:
        x1, y1, x2, y2 = x2, y2, x1, y1
    segments.append((x1, x2, y1, y2))

# Ordena os segmentos pelo início do intervalo x
segments.sort()

# Para busca binária, vamos guardar os inícios dos intervalos
x_starts = [seg[0] for seg in segments]

def find_segment(x):
    idx = bisect.bisect_right(x_starts, x) - 1
    if idx < 0 or x > segments[idx][1]:
        return None
    return segments[idx]

def simulate(x):
    while True:
        seg = find_segment(x)
        if not seg:
            return (x,)
        x1, x2, y1, y2 = seg
        # Calcula y para o x atual
        if x1 == x2:
            y = max(y1, y2)
        else:
            y = y1 + (y2 - y1) * (x - x1) / (x2 - x1)
        if y1 == y2:
            # Segmento horizontal, balão fica preso
            return (x, int(y))
        else:
            # Segmento inclinado, balão escapa pelo ponto mais alto
            if y1 > y2:
                x, y = x1, y1
            else:
                x, y = x2, y2
            # Continua a simulação a partir do novo x
            # Mas se não houver segmento cobrindo esse x, balão escapa
            if not find_segment(x):
                return (x,)
            # Se houver, repete o processo

for _ in range(C):
    qx = int(input())
    res = simulate(qx)
    print(*res)
```

Problem M

O recente terremoto em Nlogônia não chegou a afetar muito as edificações da capital, principal epicentro do abalo. Mas os cientistas detectaram que o principal dique de contenção teve um dano significativo na sua parte subterrânea que, se não for consertado rapidamente, pode causar o seu desmoronamento, com a consequente inundação de toda a capital. O conserto deve ser feito por mergulhadores, a uma grande profundidade, em condições extremamente difíceis e perigosas. Mas como é a sobrevivência da própria cidade que está em jogo, seus moradores acudiram em grande número como voluntários para essa perigosa missão. Como é tradicional em missões perigosas, cada mergulhador recebeu no início do mergulho uma pequena placa com um número de identificação. Ao terminar o mergulho, os voluntários devolviam a placa de identificação, colocando-a em um repositório. O dique voltou a ser seguro, mas aparentemente alguns voluntários não voltaram do mergulho. Você foi contratado para a penosa tarefa de, dadas as placas colocadas no repositório, determinar quais voluntários perderam a vida salvando a cidade.

Inputs

A entrada é composta de duas linhas. A primeira linha contém dois inteiros N e R ($1 \le R \le N \le 104$), indicando respectivamente o número de voluntários que mergulhou e o número de voluntários que retornou do mergulho. Os voluntários são identificados por números de 1 a N. A segunda linha da entrada contém R inteiros, indicando os voluntários que retornaram do mergulho (ao menos um voluntário retorna do mergulho).

Outputs

Seu programa deve produzir uma única linha, contendo os identificadores dos voluntários que não retornaram do mergulho, na ordem crescente de suas identificações. Deixe um espaço em branco após cada identificador (inclusive, após o último identificador). Se todos os voluntários retornaram do mergulho, imprima apenas o caractere "*" (asterisco).

Examples

Exemplo de entrada 1	Exemplo de saída 1
5 3 3 1 5	2 4

Exemplo de entrada 2	Exemplo de saída 2
6 6 6 1 3 2 5 4	*

Code

```python
# Estrutura de dados utilizada: conjuntos (set) para identificar os voluntários que não retornaram
# Tipo de algoritmo: diferença de conjuntos e ordenação

N, R = map(int, input().split())
retornaram = set(map(int, input().split()))
todos = set(range(1, N+1))
nao_retornaram = sorted(todos - retornaram)

if not nao_retornaram:
    print("*")
else:
    print(' '.join(map(str, nao_retornaram)), end=' ')
```

Problem N

Um carregamento de Nlogs, principal produto de exportação de Nlogônia, está no porto, em contêineres, pronto para ser embarcado. Todos os contêineres têm as mesmas dimensões e são cubos. Os contêineres estão organizados no pátio do porto em L linhas e C colunas, num total de LC contêineres. Cada contêiner está marcado com um número de identificação distinto, de 1 a LC. Cada uma das L linhas de contêineres será embarcada em um navio distinto. Para facilitar o desembarque nos diversos países em que serão entregues, os contêineres de uma linha devem estar organizados de forma que os números de identificação estejam ordenados. Mais precisamente, a linha 1 foi organizada no pátio de forma a conter os contêineres identificados de 1 a C ordenados crescentemente, a linha 2 de forma a conter os contêineres de C + 1 a 2C (ordenados crescentemente), e assim por diante, até a linha L, organizada de forma a conter os contêineres de (L – 1)C + 1 a LC (ordenados crescentemente).

O guindaste de embarque é capaz de movimentar ou uma linha completa ou uma coluna completa de contêineres, não sendo capaz de movimentar outros tipos de agrupamentos ou contêineres individuais. Na noite anterior ao embarque, um grupo de estivadores operou os guindastes para trocar linhas e colunas do carregamento, como forma de protestar quanto aos baixos salários. A figura (b) acima mostra a configuração dos contêineres após a troca das linhas 1 e 4; a figura (c) mostra a configuração após mais uma troca, entre as colunas 2 e 3. O carregamento precisa ser embarcado ainda hoje, mas antes disso é necessário que os contêineres sejam reorganizados da forma descrita. Você deve escrever um programa que, dada a informação sobre a posição de cada contêiner após o protesto, determine se é possível recolocar os contêineres na forma originalmente prevista utilizando apenas os guindastes, e nesse caso calcular o menor número de trocas de linhas e colunas necessário para esse fim.

Inputs

A primeira linha de um programa contém dois inteiros L ($1 \le L \le 300$) e C ($1 \le C \le 300$) indicando respectivamente o número de linhas e o número de colunas do carregamento. As L linhas seguintes descrevem a posição dos contêineres depois do protesto dos estivadores. Cada uma dessas L linhas contém C números inteiros $X_{l,c}$ ($1 \le X_{l,c} \le LC$) indicando a posição de um contêiner. Cada número inteiro entre 1 e LC aparece na entrada, em alguma das L linhas. É garantido que cada número na configuração apareça uma única vez cada e que todos os números entre 1 e LC aparecerão na mesma.

Outputs

Seu programa deve produzir uma única linha, contendo um único inteiro, o número mínimo de trocas de linhas e colunas que devem ser realizadas pelo guindaste para recolocar os contêineres na posição original. Se não for possível colocar os contêineres na posição original, utilizando apenas trocas de linhas e colunas, imprima o caractere "*".

Examples

Exemplo de entrada 1	Exemplo de saída 1
2 2	1
3 4	

Exemplo de entrada 1	Exemplo de saída 1
1 2	

Exemplo de entrada 2	Exemplo de saída 2
3 3	*
9 2 4	
5 8 7	
6 1 3	

Exemplo de entrada 3	Exemplo de saída 3
5 4	2
13 15 14 16	
5 7 6 8	
9 11 10 12	
1 3 2 4	
17 19 18 20	

Code

```python
# Estrutura de dados utilizada: listas para representar permutações de linhas e colunas
# Tipo de algoritmo: análise de permutação e contagem de ciclos (perm cycles)

def count_cycles(perm):
    visited = [False] * len(perm)
    cycles = 0
    for i in range(len(perm)):
        if not visited[i]:
            cycles += 1
            j = i
            while not visited[j]:
                visited[j] = True
                j = perm[j]
    return len(perm) - cycles

L, C = map(int, input().split())
mat = [list(map(int, input().split())) for _ in range(L)]

# Calcula permutação de linhas
row_perm = []
for i in range(L):
    # O primeiro elemento da linha i deveria ser (i * C + 1)
    val = mat[i][0]
    correct_row = (val - 1) // C
    row_perm.append(correct_row)

# Calcula permutação de colunas
col_perm = []
for j in range(C):
    # O elemento da primeira linha, coluna j, deveria ser (j + 1)
    val = mat[0][j]
    correct_col = (val - 1) % C
    col_perm.append(correct_col)

# Verifica se a matriz pode ser obtida apenas trocando linhas e colunas
possible = True
for i in range(L):
    for j in range(C):
        expected = row_perm[i] * C + col_perm[j] + 1
        if mat[i][j] != expected:
            possible = False
            break
    if not possible:
        break

if not possible:
    print("*")
else:
    print(count_cycles(row_perm) + count_cycles(col_perm))
```

Problem O

Todos devem conhecer o jogo Zerinho ou Um (em algumas regiões também conhecido como Dois ou Um), utilizado para determinar um ganhador entre três ou mais jogadores. Para quem não conhece, o jogo funciona da seguinte maneira. Cada jogador escolhe um valor entre zero ou um; há um comando (geralmente um dos competidores anuncia em voz alta "Zerinho ou... Um!"), todos os participantes mostram o valor escolhido, utilizando uma das mãos: se o valor escolhido foi um, o competidor mostra o dedo indicador estendido; se o valor escolhido foi zero, mostra a mão com todos os dedos fechados. O ganhador é aquele que tiver escolhido um valor diferente de todos os outros; se não há um jogador com valor diferente de todos os outros (por exemplo todos os jogadores escolhem zero, ou um grupo de jogadores escolhe zero e outro grupo escolhe um), não há ganhador. Alice, Beto e Clara são grandes amigos e jogam Zerinho a toda hora: para determinar quem vai comprar a pipoca durante a sessão de cinema, quem vai entrar na piscina primeiro, etc. Jogam tanto que resolveram fazem um plugin no Facebook para jogar Zerinho. Como não sabem programar, dividiram as tarefas entre amigos que sabem, inclusive você. Dados os três valores escolhidos por Alice, Beto e Clara, cada valor zero ou um, escreva um programa que determina se há um ganhador, e nesse caso determina quem é o ganhador.

Inputs

A entrada é composta de uma única linha, que contém três inteiros A, B e C, indicando respectivamente os valores escolhidos por Alice, Beto e Clara.

Outputs

Seu programa deve produzir uma única linha, contendo um único caractere. Se o vencedor é Alice o caractere deve ser "A", se o vencedor é Beto o caractere deve ser "B", se o vencedor é Clara o caractere deve ser "C" e se não há vencedor, o caractere é "*".

Examples

Exemplo de entrada 1	Exemplo de saída 1
1 1 0	C

Exemplo de entrada 2	Exemplo de saída 2
0 0 0	*

Exemplo de entrada 3	Exemplo de saída 3
1 0 0	A

Code

```python
# Estrutura de dados utilizada: lista para armazenar as escolhas
# Tipo de algoritmo: contagem de ocorrências (linear scan)

A, B, C = map(int, input().split())
valores = [A, B, C]
if valores.count(0) == 1:
    print("A" if A == 0 else "B" if B == 0 else "C")
elif valores.count(1) == 1:
    print("A" if A == 1 else "B" if B == 1 else "C")
else:
    print("*")
```

Problem P

No automobilismo é bastante comum que o líder de uma prova, em determinado momento, ultrapasse o último colocado. O líder, neste momento, está uma volta à frente do último colocado, que se torna, assim, um retardatário. Neste problema, dados os tempos que o piloto mais rápido e o piloto mais lento levam para completar uma volta, você deve determinar em que volta o último colocado se tornará um retardatário, ou seja, será ultrapassado pelo líder. Você deve considerar que, inicialmente, eles estão lado a lado, na linha de partida do circuito, ambos no início da volta de número 1 (a primeira volta da corrida); e que uma nova volta se inicia sempre depois que o líder cruza a linha de partida.

Inputs

A única linha da entrada contém dois números inteiros X e Y ($1 \leq X < Y \leq 10000$), os tempos, em segundos, que o piloto mais rápido e o piloto mais lento levam para completar uma volta, respectivamente.

Outputs

Seu programa deve produzir uma única linha, contendo um único inteiro: a volta em que o piloto mais lento se tornará um retardatário.

Examples

Exemplo de entrada 1	Exemplo de saída 1
1 10	2

Exemplo de entrada 2	Exemplo de saída 2
4 8	2

Exemplo de entrada 3	Exemplo de saída 3
5 7	4

Exemplo de entrada 4	Exemplo de saída 5
6875 7109	31

Code

```python
# Estrutura de dados utilizada: apenas variáveis inteiras (simples)
# Tipo de algoritmo: simulação matemática (cálculo direto)

X, Y = map(int, input().split())
# O líder ultrapassa o último quando k*X >= (k-1)*Y + 1
# Procuramos o menor k tal que k*X > (k-1)*Y
k = (Y - 1) // (Y - X) + 1
print(k)
```

Problem Q

Frustrado e desanimado com os resultados de sua equipe de futebol, o Super Brasileiro Clube (SBC) resolveu investir na equipe de handebol. Para melhor avaliar os atletas, os técnicos identificaram que seria útil analisar a regularidade dos jogadores. Especificamente, eles estão interessados em saber quantos jogadores fizeram gols em todas as partidas. Como o volume de dados é muito grande, eles gostariam de ter um programa de computador para realizar essa contagem.

Inputs

A primeira linha da entrada contém dois inteiros N e M ($1 \leq N \leq 100$ e $1 \leq M \leq 100$), indicando respectivamente o número de jogadores e o número de partidas. Cada uma das N linhas seguintes descreve o desempenho de um jogador: a i-ésima linha contém M inteiros Xj ($0 \leq Xj \leq 100$, para $1 \leq j \leq M$), informando o número de gols do i-ésimo jogador em cada partida.

Outputs

Seu programa deve produzir uma única linha, contendo um único inteiro, o número de jogadores que fizeram gols em todas as partidas.

Examples

Exemplo de entrada 1	Exemplo de saída 1
5 3	0
0 0 0	
1 0 5	
0 0 0	
0 1 2	
1 1 0	

Exemplo de entrada 2	Exemplo de saída 2
12 5	2
4 4 2 3 7	
0 0 0 1 0	
7 4 7 0 6	
1 2 3 3 2	
0 0 0 0 0	
4 0 9 10 10	
0 1 0 0 0	

Exemplo de entrada 2	Exemplo de saída 2
1 2 0 2 3	
10 10 10 1 0	
0 3 3 3 4	
10 10 0 10 10	
1 1 2 0 9	

Code

```python
# Estrutura de dados utilizada: lista de listas (matriz) para armazenar os gols dos jogadores
# Tipo de algoritmo: varredura sequencial (linear scan) para checar se todos os valores são maiores que zero

N, M = map(int, input().split())
count = 0
for _ in range(N):
    gols = list(map(int, input().split()))
    if all(g > 0 for g in gols):
        count += 1
print(count)
```

Problem R

Um baralho contém um número par 2n de cartas a1, a2, . . . , a2n, todas distintas (a1 < a2 < · · · < a2n). O baralho encontra-se perfeitamente ordenado, ou seja, a primeira carta é a1, a segunda carta é a2, e assim por diante, até a última carta, que é a2n. Um croupier então executa repetidamente um procedimento de embaralhar, que consiste em dois passos: 1. o baralho é dividido ao meio; 2. as cartas das duas metades são então intercaladas, de maneira que se a sequência de cartas do baralho no início do passo 1 é x1, x2, . . . , x2n, então ao final do passo 2 a sequência de cartas se torna xn+1, x1, xn+2, x2, . . . , x2n, xn. Dado o número de cartas do baralho, escreva um programa que determine quantas vezes o procedimento de embaralhar descrito acima deve ser repetido de forma que o baralho volte a ficar ordenado.

Inputs

A única linha da entrada contém um inteiro par P (2 ≤ P ≤ 2 × 105), indicando o número de cartas do baralho (note que o valor P corresponde ao valor 2n na descrição acima).

Outputs

Seu programa deve produzir uma única linha contendo um único inteiro, o número mínimo de vezes que o processo de embaralhamento deve ser repetido para que o baralho fique novamente ordenado.

Examples

Exemplo de entrada 1	Exemplo de saída 1
4	4

Exemplo de entrada 2	Exemplo de saída 2
6	3

Exemplo de entrada 3	Exemplo de saída 3
2	2

Exemplo de entrada 4	Exemplo de saída 4
100002	100002

Code

```python
# Estrutura de dados utilizada: apenas variáveis inteiras (simulação de permutação por índices)
# Tipo de algoritmo: análise de permutação cíclica (cálculo do período do embaralhamento)

P = int(input())
pos = 1
count = 0
while True:
    if pos <= P // 2:
        pos = 2 * pos
    else:
        pos = 2 * (pos - P // 2) - 1
    count += 1
    if pos == 1:
        break
print(count)
```

Problem S

Tri-Du é um jogo de cartas derivado do popular jogo de Truco. O jogo utiliza um baralho normal de 52 cartas, com treze cartas de cada naipe, mas os naipes são ignorados. Apenas o valor das cartas, considerados como inteiros de 1 a 13, são utilizados. No jogo, cada jogador recebe três cartas. As regras são simples:

- Um trio (três cartas de mesmo valor) ganha de uma dupla (duas cartas de mesmo valor).
- Um trio formado por cartas de maior valor ganha de um trio formado por cartas de menor valor.
- Uma dupla formada por cartas de maior valor ganha de uma dupla formada por cartas de menor valor.

Note que o jogo pode não ter ganhador em muitas situações; nesses casos, as cartas distribuídas são devolvidas ao baralho, que é embaralhado e uma nova partida é iniciada. Um jogador já recebeu duas das cartas que deve receber, e conhece seus valores. Sua tarefa é escrever um programa para determinar qual o valor da terceira carta que maximiza a probabilidade de esse jogador ganhar o jogo.

Inputs

A entrada consiste em uma única linha que contém dois inteiros, A ($1 \leq A \leq 13$) e B ($1 \leq B \leq 13$) indicando os valores das duas primeiras cartas recebidas.

Outputs

Seu programa deve produzir uma única linha com um inteiro representando o valor da carta que maximiza a probabilidade de o jogador ganhar a partida.

Examples

Exemplo de entrada 1	Exemplo de saída 1
10 7	10

Exemplo de entrada 2	Exemplo de saída 2
2 2	2

Code

```python
# Estrutura de dados utilizada: apenas variáveis inteiras (simples)
# Tipo de algoritmo: comparação direta (condicional simples)

A, B = map(int, input().split())
print(max(A, B))
```

Problem T

O fatorial de um número inteiro positivo N, denotado por N!, é definido como o produto dos inteiros positivos menores do que ou iguais a N. Por exemplo 4! = 4 × 3 × 2 × 1 = 24. Dado um inteiro positivo N, você deve escrever um programa para determinar o menor número k tal que N = a1! + a2! + . . . + ak!, onde cada ai, para 1 ≤ i ≤ k, é um número inteiro positivo. Por exemplo, para N = 10 a resposta é 3, pois é possível escrever N como a soma de três números fatoriais: 10 = 3! + 2! + 2!. Para N = 25 a resposta é 2, pois é possível escrever N como a soma de dois números fatoriais: 25 = 4! + 1!.

Inputs

A entrada consiste de uma única linha que contém um inteiro N (1 ≤ N ≤ 105).

Outputs

Seu programa deve produzir uma única linha com um inteiro representando a menor quantidade de números fatoriais cuja soma é igual ao valor de N.

Examples

Exemplo de entrada 1	Exemplo de saída 1
10	3

Exemplo de entrada 2	Exemplo de saída 2
25	2

Code

```python
# Estrutura de dados utilizada: lista para armazenar fatoriais
# Tipo de algoritmo: algoritmo guloso (greedy)

N = int(input())
# Pré-calcula todos os fatoriais menores ou iguais a N
fats = []
f = 1
i = 1
while f <= N:
    fats.append(f)
    i += 1
    f *= i

count = 0
i = len(fats) - 1
while N > 0:
    if fats[i] <= N:
        N -= fats[i]
        count += 1
    else:
        i -= 1
print(count)
```

Problem U

Um jogo de estratégia, com J jogadores, é jogado em volta de uma mesa. O primeiro a jogar é o jogador 1, o segundo a jogar é o jogador 2 e assim por diante. Uma vez completada uma rodada, novamente o jogador 1 faz sua jogada e a ordem dos jogadores se repete. A cada jogada, um jogador garante uma certa quantidade de Pontos de Vitória. A pontuação de cada jogador consiste na soma dos Pontos de Vitória de cada uma das suas jogadas. Dado o número de jogadores, o número de rodadas e uma lista representando os Pontos de Vitória na ordem em que foram obtidos, você deve determinar qual é o jogador vencedor. Caso mais de um jogador obtenha a pontuação máxima, o jogador com pontuação máxima que tiver jogado por último é o vencedor.

Inputs

A entrada consiste de duas linhas. A primeira linha contém dois inteiros J e R, o número de jogadores e de rodadas respectivamente ($1 \leq J, R \leq 500$). A segunda linha contém $J \times R$ inteiros, correspondentes aos Pontos de Vitória em cada uma das jogadas feitas, na ordem em que aconteceram. Os Pontos de Vitória obtidos em cada jogada serão sempre inteiros entre 0 e 100, inclusive.

Outputs

Seu programa deve produzir uma única linha, contendo o inteiro correspondente ao jogador vencedor.

Examples

Exemplo de entrada 1	Exemplo de saída 1
3 3 1 1 1 1 2 2 2 3 3	3

Exemplo de entrada 2	Exemplo de saída 2
2 3 0 0 1 0 2 0	1

Code

```python
# Estrutura de dados utilizada: lista para armazenar a pontuação de cada jogador
# Tipo de algoritmo: varredura sequencial (linear scan) e desempate pelo último a jogar

J, R = map(int, input().split())
pontos = list(map(int, input().split()))
soma = [0] * J

for i, p in enumerate(pontos):
    soma[i % J] += p

max_pontos = max(soma)
# Em caso de empate, vence o último a jogar com a pontuação máxima
for i in range(len(pontos)-1, -1, -1):
    jogador = i % J
    if soma[jogador] == max_pontos:
        print(jogador + 1)
        break
```

Problem V

Um investidor principiante deseja aprender a investir na bolsa de valores. Como ele não tem experiência, selecionou uma única empresa, e acompanhou os valores diários das ações dessa empresa, durante N dias. Ficou curioso quanto teria ganhado se tivesse investido nesse período em que acompanhou os valores. Na verdade, o investidor é milionário e tem muito dinheiro, suficiente para comprar qualquer quantidade de ações da empresa. Entretanto, como é um investidor cuidadoso, decidiu que nunca teria mais do que uma ação da empresa. Como sempre há intermediários, a corretora de valores cobra uma taxa fixa de C reais a cada compra de uma ação da empresa. Você deve calcular qual o lucro máximo que o investidor poderia ter auferido, investindo durante alguns dos N dias, podendo inclusive decidir não investir.

Inputs

A primeira linha contém dois inteiros, N e C ($1 \leq N \leq 2 \times 105$ e $0 \leq C \leq 30$). A segunda linha contém as N cotações P1, P2, . . . ,PN, dos dias 1, 2, . . . , N, respectivamente. Cada cotação Pi satisfaz as desigualdades $1 \leq Pi \leq 1000$.

Outputs

Seu programa deve produzir uma única linha com um inteiro representando o lucro máximo do investidor, em reais.

Examples

Exemplo de entrada 1	Exemplo de saída 1
6 10	20
100 120 130 80 50 40	

Exemplo de entrada 2	Exemplo de saída 2
5 10	0
70 80 50 40 50	

Exemplo de entrada 2	Exemplo de saída 2
13 30	220
10 80 20 40 30 50 40 60 50 70 60 10 200	

Code

```python
# Estrutura de dados utilizada: variáveis para rastrear estado de compra/venda
# Tipo de algoritmo: varredura sequencial (linear scan) com simulação de compra/venda

N, C = map(int, input().split())
P = list(map(int, input().split()))

lucro = 0
comprado = False
preco_compra = 0

for i in range(N):
    # Comprar se não está comprado e amanhã o preço sobe o suficiente
    if not comprado and i < N-1 and P[i+1] > P[i] + C:
        comprado = True
        preco_compra = P[i] + C
    # Vender se está comprado e amanhã o preço cai ou é o último dia
    elif comprado and (i == N-1 or P[i+1] < P[i]):
        lucro += P[i] - preco_compra
        comprado = False

print(lucro)
```

Problem W

Um Kit de Encolhimento de Polígonos é um material muito utilizado nas aulas de magia geométrica na Nlogônia. O kit consiste de dois pontos, A e B no plano cartesiano. Considere um polígono convexo dado pelos vértices 1, 2...N, nessa ordem. Para encolher esse polígono usando o kit, algumas regras devem ser respeitadas. Cada vértice x do polígono deve ser movido uma vez só: para o ponto médio do segmento Ax ou para o ponto médio do segmento Bx. A operação de encolhimento deve produzir um novo polígono convexo que preserve a ordem relativa dos vértices do polígono original. Em outras palavras, considerando todas as possíveis maneiras de aplicar o kit, apenas aquelas cuja sequência final dos vértices 1, 2...N representa um polígono convexo são válidas. Veja que o polígono convexo original pode estar em sentido horário e uma operação de encolhimento válida produzir um polígono convexo em sentido anti-horário, na mesma ordem dos vértices. Apenas a ordem relativa dos pontos é importante, não o sentido. E sabido que magia geométrica não é o forte da maioria dos alunos. A professora pediu que eles usassem o kit de encolhimento para encolher um polígono convexo fornecido por ela de forma a obter a menor área possível e um amigo seu implorou para que você resolva a questão por ele. Responda a menor área possível do polígono para ele.

Considerando um uso válido do kit, onde o polígono sombreado é o de menor área possível que preserva a sequência dos vértices. Os pontos A e B correspondem aos pontos do kit. Note que, apesar do nome encolhimento, às vezes é possível utilizar o kit para aumentar a área dos polígonos! Como geometria é difícil! Observe que um único ponto ou uma reta não são considerados polígonos. Sendo assim, se um uso do kit produzir como resultado algo diferente de um polígono convexo, esse não é um uso válido.

Inputs

A primeira linha da entrada contém um inteiro N (3 □ N □ 105), o número de vértices do polígono. Seguem N linhas, cada uma com dois inteiros x, y (-106 □ x, y □ 106), os vértices do polígono. A última linha da entrada contém quatro inteiros, Ax, Ay, Bx e By (-106 □ Ax, Ay, Bx, By □ 106), as coordenadas x e y de A e as coordenadas x e y de B, respectivamente. Os pontos da entrada serão dados na ordem correta em que aparecem no polígono, no sentido horário ou anti-horário. Não haverá pontos repetidos e o polígono será convexo.

Outputs

Seu programa deve produzir uma linha, contendo um número real, com 3 casas decimais de precisão, representando a menor área possível para um polígono obtido com o uso do kit.

Examples

Exemplo de entrada 1	Exemplo de saída 1
3	3.500
20 6	
4 8	
2 6	

Exemplo de entrada 1	Exemplo de saída 1
0 0 4 0	

Exemplo de entrada 2	Exemplo de saída 2
3	1.000
0 4	
4 4	
0 0	
3 -2 -3 -2	

Exemplo de entrada 3	Exemplo de saída 3
3	2.000
0 4	
4 4	
0 0	
2 -2 -2 -2	

Code

```python
# Estrutura de dados utilizada: variáveis para rastrear estado de compra/venda
# Tipo de algoritmo: varredura sequencial (linear scan) com simulação de compra/venda

N, C = map(int, input().split())
P = list(map(int, input().split()))

lucro = 0
comprado = False
preco_compra = 0

for i in range(N):
    # Comprar se não está comprado e amanhã o preço sobe o suficiente
    if not comprado and i < N-1 and P[i+1] > P[i] + C:
        comprado = True
        preco_compra = P[i] + C
    # Vender se está comprado e amanhã o preço cai ou é o último dia
    elif comprado and (i == N-1 or P[i+1] < P[i]):
        lucro += P[i] - preco_compra
        comprado = False

print(lucro)
```